

Spatial Vision & Localization

Welcome to the next phase of computer vision. In the previous module, you learned how to treat an image as an array of pixels and track colors. Now, we are bridging the gap between the digital screen and the physical world.

To make a robot navigate autonomously, it needs to know exactly how far away objects are in centimeters, not just pixels. Read the below information, interpret them properly and dive to the task2.

1. Camera Calibration

The Concept: No physical camera lens is perfect. Lenses introduce distortion—often making straight lines look curved at the edges of an image like a fish-eye effect as shown in the figure below.



If you try to calculate the distance to an object using a distorted image, your math will be completely wrong.

Calibration is the process of taking multiple pictures of a perfectly rigid, known geometric pattern like a checkerboard from different angles. OpenCV uses these images to mathematically model the flaws in the lens and calculate the **Camera Matrix** and **Distortion Coefficients**. Once you have these, you can "undistort" any future images taken by that camera.

The Code You Need:

Find the corners on a grayscale checkerboard image

```
ret, corners = cv2.findChessboardCorners(gray_image, (columns, rows),  
None)
```

Calibrate to get the Camera Matrix (mtx) and Distortion Coefficients (dist)
 objpoints: 3D real-world coordinates of the checkerboard (which you define)
 imgpoints: 2D pixel coordinates found by findChessboardCorners

```
ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(objpoints, imgpoints,
gray_shape, None, None)
```

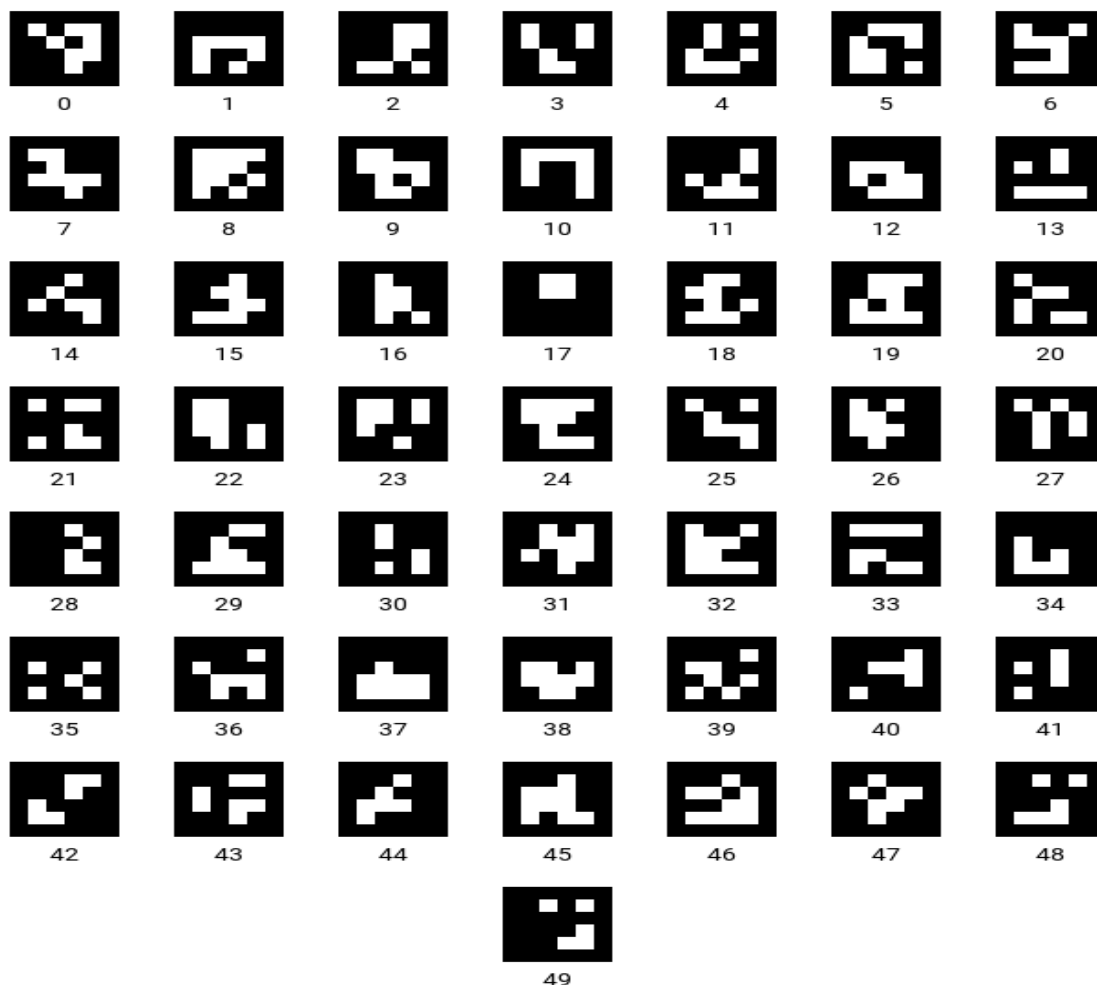
Fix a distorted image

```
fixed_image = cv2.undistort(raw_image, mtx, dist, None, mtx)
```

2. ArUco Marker Detection & Pose Estimation

The Concept: ArUco markers are synthetic, square barcodes designed specifically for computer vision. Because we know they are perfectly square and we know their exact physical size (e.g., 5.0 cm), they act as anchoring points.

Aruco markers 4x4 50



Once you detect the four corners of a marker in an image, you can use a math algorithm called **Perspective-n-Point (PnP)**. By feeding the 2D pixel corners and the 3D physical size into `solvePnP`, OpenCV instantly calculates the marker's 3D position (Translation Vector or `tvec`) and its 3D rotation (Rotation Vector or `rvec`) relative to the camera lens.

The Code You Need:

Set up the detector

```
dictionary = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
parameters = cv2.aruco.DetectorParameters()
detector = cv2.aruco.ArucoDetector(dictionary, parameters)
```

Find the markers in the image

```
corners, ids, rejected = detector.detectMarkers(image)
```

Estimate the Pose (3D position) of the marker

marker_3d_edges: The physical coordinates of the marker's 4 corners (e.g., in cm)

mtx, dist: The camera matrix and distortion coefficients from calibration

```
success, rvec, tvec = cv2.solvePnP(marker_3d_edges, corners[0][0], mtx,
dist)
```

`tvec[2][0]` will give you the Z distance (depth) from the camera!

`tvec[0][0]` will give you the X offset (left/right) from the camera center!

3. Perspective Transformation (Homography)

The Concept: If a camera is mounted on a wall looking down at a robot arena, the far side of the arena will look smaller than the near side due to perspective. This makes measuring distances difficult.

Perspective transformation fixes this by warping the angled image into a perfectly flat, top-down "bird's-eye view." You pick 4 specific points in your angled image, and tell OpenCV where those 4 points *should* be in a perfect square. OpenCV calculates a 3x3 Homography Matrix that stretches the image to fit.

The Code You Need:

Define your 4 source points (from the angled image)

```
pts_src = np.float32([[x1, y1], [x2, y2], [x3, y3], [x4, y4]])
```

Define your 4 destination points (where you want them to go)

Example: Mapping to a perfect 500x500 square image

```
pts_dst = np.float32([[0, 0], [500, 0], [500, 500], [0, 500]])
```

Calculate the Transformation Matrix

```
matrix = cv2.getPerspectiveTransform(pts_src, pts_dst)
```

Warp the image

```
flat_image = cv2.warpPerspective(angled_image, matrix, (500, 500))
```

4. Coordinate Transformation & Extraction

The Concept: Once an image is perfectly flat (bird's-eye view), extracting coordinates is just a matter of finding the center of an object in pixels, and applying a mathematical scale to convert it to real-world centimeters.

To find the center of an irregularly shaped colored object (like a blob of green), we use **Image Moments**. A moment is a weighted average of the image pixels' intensities, which easily lets us calculate the exact center of mass (centroid) of a contour.

The Code You Need:

Find the center (centroid) of a contour using Moments

```
M = cv2.moments(largest_contour)
if M["m00"] != 0:
    cx = int(M["m10"] / M["m00"]) # X pixel coordinate
    cy = int(M["m01"] / M["m00"]) # Y pixel coordinate
```

Linear Scaling (Pixels to Centimeters)

If your 500x500 pixel image represents a 200cm x 200cm physical arena...

Scale factor = Physical Size / Pixel Size = 200 / 500 = 0.4

```
x_cm = cx * 0.4
y_cm = cy * 0.4
```

Resources:-

Aruco detection:

<https://www.geeksforgeeks.org/computer-vision/detecting-aruco-markers-with-opencv-and-python-1/>

▶ ArUco Marker Pose Estimation and Detection in Real-Time using OpenCV Python

Camera calibration:

<https://www.geeksforgeeks.org/python/camera-calibration-with-python-opencv/>

▶ OpenCV Python Camera Calibration (Intrinsic, Extrinsic, Distortion)

Coordinate transform:

<https://www.geeksforgeeks.org/computer-vision/mapping-coordinates-from-3d-to-2d-using-opencv-python/>

<https://motion.cs.illinois.edu/RoboticSystems/CoordinateTransformations.html>

▶ Coordinate Transformations - How robots move through space

▶ Coordinate Transformations in 2D | Mapping | Robotics 101

▶ Coordinate Transformations in 2D (Part 2) | Mapping | Robotics 101

Perspective transform:

<https://www.geeksforgeeks.org/python/perspective-transformation-python-opencv/>

▶ Perspective Transformation | OpenCV in Python | Image Processing